מספר התלמיד הנבחן
רשום את כל תשע הספרות

הדבק כאן את מדבקת הנבחן

האוניברסיטה הפתוחה

י"ח באב תשע"ט

מס' שאלון - 517

19 באוגוסט 2019

סמסטר 2019ב

מס' מועד 91

4 / 20905

שאלון בחינת גמר

20905 - שפות תכנות

משך בחינה: 3 שעות

בשאלון זה **15** עמודים

מבנה הבחינה:

בבחינה ארבע שאלות.

עליכם לענות על כל השאלות.

משקל השאלות מפורט בגוף השאלות.

שימו לב:

את תשובותיכם רשמו אך ורק במחברת הבחינה.

כתבו בכתב יד ברור ומסודר.

חומר עזר:
ספר הקורס: essentials of programming languages
מדריך הלמידה. מותרות הערות בכתב יד, ע"ג הספרים.
אין להכניס חומר מודפס או כל חומר אחר מכל סוג שהוא.

בהצלחה !!!

אינכם חייבים
להחזיר את השאלון לאוניברסיטה הפתוחה

שאלה 1 (25 נקודות)

מעוניינים להרחיב את שפת LET (שפת "ויהי") עם פעולות על זוג סדור.

להלן הדקדוק המגדיר את התחביר החדש של הפעולות החדשות:

Expression* ::=** make-pair (Expression*** , ***Expression***)

make-pair-exp (exp1 exp2)

Expression* ::=** car-pair (Expression***)

car-pair-exp (exp1)

Expression* ::=** cdr-pair (Expression***)

cdr-pair-exp (exp1)

שלושת הביטויים החדשים מחקים את הפעולות cons,car,cdr הקיימות בשפת scheme.
במקרה של שימוש שגוי בהם יש להדפיס הודעת שגיאה מתאימה.

להלן דוגמא להמחשת השימוש בביטויים החדשים:

```
> (run "let pr1=make-pair(10,make-pair( -(20,5), zero? (5) ))
in cdr-pair (cdr-pair(pr1))")
(bool-val #f)
> |
```

דוגמה נוספת:

```
> (run "let pr1=make-pair(10,make-pair( -(20,5), zero? (5) ))
in pr1")
(pair-val (num-val 10) (pair-val (num-val 15) (bool-val #f)))
>
```

א. (3 נקודות)

בשאלה זו יש טיפוס נתונים חדש (זוג סדור) ולכן יש לעדכן את הגדרות השפה בהתאם. הזוג הסדור יכול להכיל בתוכו איברים מכל אחד מהטיפוסים הנתמכים בשפה, כלומר, מספר, ערך בוליאני או זוג סדור.

הוסיפו לשפה טיפוס נתונים חדש בשם pair-val (בדומה ל- num-val ו bool-val) הסבירו במחברת הבחינה מהו השינוי והיכן הוא נכתב.

לנוחיותכם, להלן ההגדרה של expval :

```
(define-datatype expval expval?
  (num-val
    (value number?))
  (bool-val
    (boolean boolean?)))
```

ב. (4 נקודות)

להלן נתונים ה-scanner, וה-parser של שפת "ויהי". כתבו מהו השינוי הדרוש בדקדוק של השפה כך שיתאים להגדרת הדקדוק החדש. **<u>את תשובותיכם כתבו אך ורק במחברת הבחינה.</u>**

ה-scanner:

```
(define the-lexical-spec
  '((whitespace (whitespace) skip)
   (comment ("%" (arbno (not #\newline))) skip)
   (identifier (letter (arbno (or letter digit "_" "-" "?"))) symbol)
   (number (digit (arbno digit)) number)
   (number ("-" digit (arbno digit)) number)
  ))
```

ה-parser:

```
(define the-grammar
  '((program (expression) a-program)
   (expression (number) const-exp)
   (expression ("-" "(" expression "," expression ")") diff-exp)
   (expression ("zero?" "(" expression ")") zero?-exp)
   (expression ("if" expression "then" expression "else" expression) if-exp)
   (expression (identifier) var-exp)
   (expression ("let" identifier "=" expression "in" expression) let-exp)
  ))
```

ג. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"ויהי"** כפי שמופיעה בספר הלימוד. יש לעדכנה כך שתחשב את הביטויים החדשים על פי הדקדוק שהוגדר עבורם. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "ויהי".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

הפרוצדורה *value-of* (כפי שמופיעה בספר בעמוד 71):

```
;;value-of  :  Exp x Env ----> ExpVal
 (define value-of
  (lambda (exp env)
   (cases expression exp
    (const-exp (num) (num-val num))
    (var-exp (var) (apply-env env var))
    (diff-exp (exp1 exp2)
       (let ((val1 (value-of exp1 env))
             (val2 (value-of exp2 env)))
         (let ((num1 (expval->num val1))
               (num2 (expval->num val2)))
           (num-val (- num1 num2)))))
    (zero?-exp (exp1)
      (let ((val1 (value-of exp1 env)))
        (let ((num1 (expval->num val1)))
          (if (zero? num1) (bool-val #t)  (bool-val #f)))))
    (if-exp (exp1 exp2 exp3)
      (let ((val1 (value-of exp1 env)))
        (if (expval->bool val1) (value-of exp2 env) (value-of exp3 env))))
    (let-exp (var exp1 body)
      (let ((val1 (value-of exp1 env)))
         (value-of body (extend-env var val1 env))))
    )))
```

שאלה 2 (25 נקודות)

שאלה זו עוסקת בשדרוג של שפת "וישגור" (שפת PROC) המתוארת בספר הלימוד בפרק 3 בעמודים 74-81. שפה זו מאפשרת הגדרה והפעלה של פרוצדורות לא רקורסיביות.

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם call-version-exp.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression ::= call-version *number* of *identifier* with *Expression*

Call-version-exp (vnum prc param)

לביטוי **call-version-exp** המרכיבים הבאים:

- **vnum** – מספר הגרסה של הפרוצדורה שמעוניינים להפעיל
- **prc** – שם הפרוצדורה שרוצים להפעיל (לא ניתן לבצע ביטוי זה על פרוצדורה אנונימית)
- **param** – ביטוי שערכו משמש כפרמטר לפרוצדורה.

הבהרה: אם בסביבה נתונה מופיעים 3 הגדרות של פרוצדורה כלשהי, נאמר על ההגדרה הראשונה שהיא גרסה מספר 1, על ההגדרה השניה שהיא גרסה מספר 2, ועל ההגדרה השלישית שהיא גרסה מספר 3.

בשאלה זו מעוניינים לממש יכולת של הפעלת גרסה מסוימת של פרוצדורה מבין מספר גרסאות שלה המאוחסנות בסביבה הנתונה.

ביטוי call-version-exp אמור להפעיל פרוצדורה, אך בניגוד לביטוי call-exp המפעיל פרוצדורה על פי הגדרתה האחרונה בסביבה (בסדר כרונולוגי), ביטוי call-version-exp צריך להפעיל גרסה מסוימת של הפרוצדורה (במקרה שיש לה מספר הגדרות בסביבה הנתונה, אחרת זו שגיאה)

להלן דוגמא :

```
let p=30
in
 let p= proc (x) -(x,8)
 in
  let p=proc (t) zero? (-(t,3))
  in
   let p=7
   in
    let p= proc (w) -(w,-1)
    in
     -(call-version 1 of p with 20 , (p 20))
```

בדוגמה זו , ישנו מבנה של let מקונן. התשובה המוחזרת מדוגמה זו היא ערכו של הביטוי הכתוב בשורה האחרונה, ביטוי זה מחושב עם סביבה המכילה בתוכה 5 הגדרות ל-p, שתיים מהן הם מספר, ושלוש האחרות הגדרות שונות של פרוצדורות ששמן p. הפרוצדורה p המוגדרת בביטוי ה-let השני היא גרסה מספר 1 של הפרוצדורה p, הפרוצדורה p המוגדרת בביטוי ה- let השלישי היא גרסה מספר 2 של הפרוצדורה p, והפרוצדורה p המוגדרת בביטוי ה-let האחרון היא גרסה מספר 3 של p.

הדוגמה מחזירה כתשובה תוצאת פעולת חיסור בין הפעלת גרסה מספר 1 של p עם פרמטר 20, לבין ההפעלה הרגילה של פרוצדורות הקיימת בשפת וישגור, המפעילה את המופע האחרון של p בסביבה (בהנחה שהוא קיים ושהוא פרוצדורה, אחרת תהיה שגיאה).

במקרה זה, הפעלת גרסה מספר 1 של p (ואכן קיימת גרסה כזו, כי ל-p יש 3 גרסאות שונות) עם פרמטר 20, תחזיר 12.

הביטוי (p 20) הוא הפעלה רגילה כפי שהוגדרה בשפת "וישגור " של פרוצדורה p. וזו תגרום להפעלת ההגדרה האחרונה של p בסביבה, שהיא בעצם גרסה מספר 3, וזו תחזיר תשובה 21

לכן התשובה שתוחזר מכלל הביטוי בדוגמה היא תוצאת החישוב של 12-21 שהיא (num-val -9)

להלן דוגמה נוספת:

```
let p=10
in
 let p= proc(x)  -(x,1)
 in
  call-version 2 of p with 80
```

דוגמה זו אמורה להדפיס הודעת שגיאה בעת החישוב, מאחר ולפרוצדורה p אין גרסה מספר 2 אלא רק גרסה מספר 1.

(המשך השאלה בעמוד הבא)

להלן דוגמה נוספת:

```
let p=proc (t) zero? (-(t,3))
  in
   let p=7
   in
    let p= proc (w) -(w,-1)
    in
     let p= 100
     in
      -(call-version 1 of p with 20 , (p 20))
```

גם דוגמה זו אמורה להדפיס הודעת שגיאה בזמן החישוב מאחר ובביטוי (p 20) ה-p המוזכר בביטוי אינו פרוצדורה אלא הערך 100.

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וישגור**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
      (define the-grammar
          '((program (expression) a-program)

            (expression (number) const-exp)
            (expression
              ("-" "(" expression "," expression ")")
      diff-exp)

            (expression
             ("zero?" "(" expression ")")
      zero?-exp)

            (expression
             ("if" expression "then" expression "else" expression)
      if-exp)

            (expression (identifier) var-exp)

            (expression
             ("let" identifier "=" expression "in" expression)
      let-exp)
```

```
      (expression
       ("proc" "(" identifier ")" expression)
proc-exp)

      (expression
       ("(" expression expression ")")
call-exp)
```

```
(expression
```

את התשובה יש לכתוב במחברת הבחינה

```
call-version-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורה *value-of* של שפת **"וישגור"** . ולאחריה פרוצדורת העזר apply-procedure כפי שמופיעה בספר הלימוד בעמוד 79. יש להשלים את החסר בפרוצדורה value-of. **<u>את התשובה כיתבו במחברת הבחינה</u>**.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וישגור".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;;value-of : Exp x Env→ExpVal
(define value-of
(lambda (exp env)
(cases expression exp
(const-exp (num) (num-val num))
(var-exp (var) (apply-env env var))
(diff-exp (exp1 exp2)
  (let ((val1 (value-of exp1 env))
(val2 (value-of exp2 env)))
(let ((num1 (expval->num val1))
   (num2 (expval->num val2)))
(num-val(- num1 num2)))))
(zero?-exp (exp1)
(let ((val1 (value-of exp1 env)))
(let ((num1 (expval->num val1)))
(if (zero? num1)(bool-val #t)(bool-val #f)))))
(if-exp (exp1 exp2 exp3)
 (let ((val1 (value-of exp1 env)))
 (if (expval->bool val1)
(value-of exp2 env)
(value-of exp3 env))))
       (let-exp (var exp1 body)
 (let ((val1 (value-of exp1 env)))
 (value-of body(extend-env var val1 env))))

     (proc-exp (var body)
          (proc-val (procedure var body env)))

(call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
                (arg (value-of rand env)))
            (apply-procedure proc arg)))
```

```
(call-version-exp  (_______________________________)

                 את התשובה יש לכתוב במחברת הבחינה
                                                  ))))
```

```
;; apply-procedure : Proc * ExpVal ->ExpVal
;; Page: 79
  (define apply-procedure
    (lambda (proc1 val)
      (cases proc proc1
        (procedure (var body saved-env)
          (value-of body (extend-env var val saved-env))))))
```

שאלה 3 (25 נקודות)

בספר הלימוד בעמודים 113-120 מתוארת שפת "**וירמוז**"(IMPLICIT-REFS).

ברצוננו להרחיב שפה זו עם ביטוי חדש בשם swap-exp.

להלן הדקדוק המגדיר את הביטוי החדש שברצוננו להוסיף:

Expression* ::=** swap (identifier* , *identifier***)

swap-exp (id1 id2)

לביטוי **swap-exp** המרכיבים הבאים:

- **id1** – שם של משתנה ראשון
- **id2** – שם של משתנה שני

ביטוי swap-exp אמור להחליף בין ערכי המשתנים הנתונים ע"י id1,id2

להלן דוגמה:

```
let p1=10
in
  let p2= proc(x) -(x,4)
   in begin
        swap(p1,p2);
        (p1 10)
      end
```

בדוגמה זו, מוגדרים 2 משתנים p1 ו-p2 הדוגמה מחליפה בין ערכיהם. ולכן מעתה p1 היא פרוצדורה ולא המספר 10. ולכן תוצאת כל הביטוי היא (num-val 6)

(המשך השאלה בעמוד הבא)

א. (7 נקודות)

להלן נתונים ה-**scanner** וה-**parser** של שפת "**וירמוז**". עליכם להשלים את החסר בריבוע המסומן.**<u>את התשובה כיתבו במחברת הבחינה</u>**.

ה-scanner:

```
(define the-lexical-spec
'((whitespace (whitespace) skip)
(comment ("%" (arbno (not #\newline))) skip)
  (identifier(letter (arbno (or letter digit "_" "-" "?")))
           symbol)
  (number (digit (arbno digit)) number)
(number ("-" digit (arbno digit)) number)
))
```

ה-parser:

```
(define the-grammar
'((program (expression) a-program)
  (expression (number) const-exp)
  (expression ("-" "(" expression "," expression ")")
    diff-exp)
  (expression ("zero?" "(" expression ")") zero?-exp)
  (expression
   ("if" expression "then" expression "else" expression)
     if-exp)
  (expression (identifier) var-exp)
  (expression ("let" identifier "=" expression "in" expression)
     let-exp)
  (expression ("proc" "(" identifier ")" expression)
     proc-exp)
  (expression ("(" expression expression ")") call-exp)
  (expression
        ("letrec"
          (arbno identifier "(" identifier ")" "=" expression)
           "in" expression)
letrec-exp)
(expression ("set" identifier "=" expression)
    assign-exp)
(expression (
```

את התשובה כיתבו במחברת הבחינה

```
)
  swap-exp)))
```

ב. (18 נקודות)

להלן הפרוצדורות *value-of* ו-*apply-procedure* של שפת **"וירמוז".**

הסבירו במדויק מה הם השינויים והתוספות הנדרשים לשילוב הביטוי החדש בשפה זו.

<u>את התשובה כיתבו במחברת הבחינה</u>.

הערות:

- המנשק לעבודה עם סביבה הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- המנשק לעבודה עם החסן (store) הוא אותו מנשק כפי שהוגדר עבור שפת "וירמוז".
- במסגרת הפתרון, ניתן לכתוב פונקציות עזר במקרה הצורך.

```
;; value-of : Exp * Env ->ExpVal
;; Page: 118, 119
  (define value-of
    (lambda (exp env)
      (cases expression exp
        (const-exp (num) (num-val num))

       (var-exp (var) (deref (apply-env env var)))

        (diff-exp (exp1 exp2)
          (let ((val1 (value-of exp1 env))
                (val2 (value-of exp2 env)))
            (let ((num1 (expval->num val1))
                  (num2 (expval->num val2)))
              (num-val
                (- num1 num2)))))

        (zero?-exp (exp1)
          (let ((val1 (value-of exp1 env)))
            (let ((num1 (expval->num val1)))
              (if (zero? num1)
                (bool-val #t)
                (bool-val #f)))))

        (if-exp (exp1 exp2 exp3)
          (let ((val1 (value-of exp1 env)))
            (if (expval->bool val1)
              (value-of exp2 env)
              (value-of exp3 env))))

        (let-exp (var exp1 body)
          (let ((v1 (value-of exp1 env)))
            (value-of body
              (extend-env var (newref v1) env))))

        (proc-exp (var body)
          (proc-val (procedure var body env)))

        (call-exp (rator rand)
          (let ((proc (expval->proc (value-of rator env)))
```

```
                (arg (value-of rand env)))
            (apply-procedure proc arg)))


      (letrec-exp (p-names b-vars p-bodies letrec-body)
          (value-of letrec-body
            (extend-env-rec* p-names b-vars p-bodies env)))

         (assign-exp (var exp1)
           (begin
             (setref!
               (apply-env env var)
               (value-of exp1 env))
             (num-val 27))))))


(define apply-procedure
      (lambda (proc1 val)
         (cases proc proc1
            (procedure (var body saved-env)
                (value-of body
                    (extend-env var (newref val) saved-env))))))
```

המשך הבחינה בעמוד הבא

שאלה 4 (25 נקודות)

להלן נתונה תוכנית בשפת "ויקיש" (INFERRED):

```
(if  (proc (x:?) zero? (-(x,7))  5)
 then  proc (z:?)  -(z,1)
 else  proc (w:?) –(w,3)
9)
```

ברצוננו לקבוע מהו טיפוס התוכנית (אם קיים).לשם כך, עליכם להשתמש באלגוריתם שנלמד בפרק 7 להקשת הטיפוס.

א. (10 נקודות)

הקצו לביטוי הנתון בשאלה ולתתי הביטויים שלו משתני-טיפוס (Type Variables) מתאימים, וחברו משוואות מתאימות לביטוי הנתון ולתתי הביטויים שלו.

ב. (15 נקודות)

פתרו את המשוואות שהרכבתם בסעיף א' תוך שימוש ב-substitution ו-unification בדומה למתואר בספר בעמודים 252-258. בסיום פתרון המשוואות רשמו מהו הטיפוס שאותו הקשתם עבור התוכנית הנתונה.

ב ה צ ל ח ה !